Plano de projeto – Simulador 2.5D de Circuitos GEII

Nov 9, 2026 · @Verônica Cutala Fernanda Capamba

Resumo

Recomendação: fazer o simulador como **aplicação web em TypeScript**, com React para a interface e Three.js (via React Three Fiber) para a cena 2.5D. Mais tarde, a mesma aplicação pode virar um programa desktop com Tauri e chegar à realidade virtual com WebXR.

O objetivo do dossier é um simulador pedagógico: zona livre tipo LTspice, componentes com aspeto 3D, motor de cálculo DC fiável e evolução até 3D completo e VR. A maior parte do trabalho é um **editor** (menus, painéis, arrastar, ligar fios) e um **motor de cálculo**. As duas coisas fazem-se muito bem em código web puro.

Esta escolha também encaixa no método de trabalho: tudo é texto e código. Assim, o Claude Code pode escrever, testar e corrigir quase tudo diretamente no repositório GitHub. Com Unity, grande parte do trabalho (cenas, prefabs, Canvas) é feita à mão no editor, onde a IA ajuda muito menos.

Web ou desktop

Escolha: **web primeiro, desktop depois (opcional)**. O mesmo código serve as duas versões, por isso não é preciso decidir já entre uma e outra.

Critério	Web (React + Three.js)	Desktop Unity (C#)	Desktop Tauri (web empacotado)
Partilhar com professores e colegas	Um link, nada a instalar	Instalador por SO, ou build WebGL pesado	Instalador leve
Interface tipo editor (painéis, menus, formulários)	Muito forte (React)	Trabalhoso (Canvas UI)	Igual ao web
Cena 3D / 2.5D	Bom (Three.js)	Excelente	Igual ao web
Realidade virtual (fase futura)	Possível (WebXR)	Excelente	Limitado

Critério	Web (React + Three.js)	Desktop Unity (C#)	Desktop Tauri (web empacotado)
Ajuda da IA (Claude Code)	Quase tudo é código: a IA faz muito	Muito trabalho manual no editor	Igual ao web
Peso das ferramentas	Leve (VS Code + Node.js)	Unity Hub + editor, vários GB	Leve + Rust
Testes automáticos do solver	Simples (Vitest)	Possível, mais pesado	Igual ao web

O ponto fraco do web é a VR: o Unity é mais maduro nisso. Mas o dossier diz que a VR só vem depois de um MVP estável. Se um dia for mesmo preciso, o motor de cálculo em TypeScript pode ser reutilizado ou reescrito em C# a partir de testes já validados.

Linguagens e frameworks

Uma só linguagem para tudo: **TypeScript** (JavaScript com tipos). Os tipos evitam muitos erros no modelo de dados (componentes, terminais, nós) e ajudam a IA a escrever código correto.

Parte do projeto	Ferramenta	Papel
Linguagem	TypeScript	Todo o código: interface, cena, motor de cálculo
Interface	React	Barra superior, menu de componentes, painéis de propriedades e resultados
Estilo	Tailwind CSS	Layout rápido e responsivo, painéis recolhíveis
Cena 2.5D	Three.js + React Three Fiber + drei	Câmara ortográfica, grelha, modelos 3D, luzes, zoom e pan
Estado da aplicação	Zustand (+ zundo para desfazer/refazer)	Lista de componentes, fios, seleção, histórico
Motor de cálculo	TypeScript puro (sem biblioteca)	Análise nodal modificada (MNA), eliminação de Gauss
Validação de ficheiros	Zod	Verifica o JSON ao abrir um projeto

Parte do projeto	Ferramenta	Papel
Build e servidor local	Vite	Arranque rápido, recarga instantânea
Testes unitários	Vitest	Testar o solver com circuitos conhecidos
Testes de interface	Playwright	Simular um utilizador: arrastar, ligar, simular
Qualidade do código	ESLint + Prettier	Estilo comum para todo o grupo
Modelos 3D	Blender → glTF/GLB	Componentes low-poly exportados para a web
Desktop (opcional)	Tauri	Transformar a app web num programa Windows/macOS/Linux
VR (futuro)	WebXR (@react-three/xr)	Laboratório virtual com headset

Porquê escrever o solver à mão? Porque é um projeto GEII: o grupo tem de saber explicar a análise nodal. Mais tarde, para circuitos complexos, dá para comparar os resultados com o ngspice, que também existe compilado para a web.

Ambientes de desenvolvimento

Ambiente principal: **VS Code + Claude Code + Node.js + Git**. É tudo gratuito e funciona em Windows, macOS e Linux.

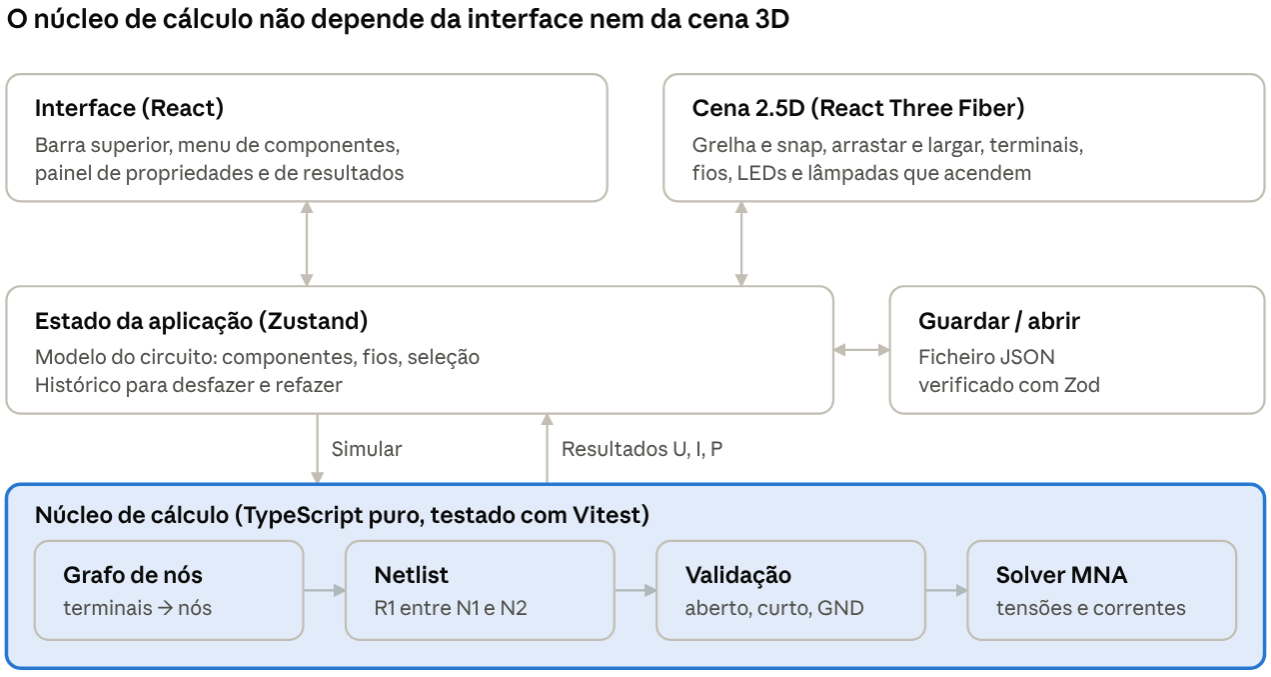
Ferramenta	Para quê	Notas
Visual Studio Code	Editor de código	Extensões: ESLint, Prettier, Tailwind CSS IntelliSense, Vitest, GitLens, Claude Code
Claude Code	Agente de IA que escreve e testa código no projeto	No terminal ou na extensão do VS Code
Node.js (versão LTS) + pnpm	Executar o Vite, os testes e instalar bibliotecas	pnpm é mais rápido que npm; npm também serve
Git + conta GitHub	Guardar versões e trabalhar em grupo	GitHub Desktop é opcional, para quem prefere botões

Ferramenta	Para quê	Notas
Google Chrome ou Edge	Testar a app	DevTools + extensão React Developer Tools
Blender	Modelar os componentes 3D	Exportar em .glb
GitHub Copilot (opcional)	Autocompletar no VS Code	Gratuito com o GitHub Student Developer Pack

Não é preciso instalar tudo já. O prompt no fim deste documento pede ao Claude Code para verificar o teu computador e dizer exatamente o que falta, com os comandos.

Arquitetura do software

Regra principal: o motor de cálculo fica separado de tudo o resto. Ele recebe dados (componentes e ligações) e devolve números. Assim pode ser testado sozinho, sem abrir a interface.



arquitetura do simulador · 4 camadas

A interface e a cena só leem e alteram o estado; ao clicar em Simular, o estado passa pelo núcleo e recebe os resultados de volta.

Organização das pastas proposta para o repositório:

```
simulador-geii/
├─ src/
│  ├─ core/            # núcleo puro, sem React
│  │  ├─ model/        # tipos: Component, Terminal, Wire, Project
│  │  ├─ library/      # catálogo de componentes (dados)
│  │  ├─ graph/        # terminais ligados → nós elétricos
│  │  ├─ netlist/
│  │  ├─ validation/
│  │  └─ solver/       # MNA, Gauss; depois díodo, AC, transiente
│  ├─ store/           # Zustand: estado + desfazer/refazer
│  ├─ scene/           # React Three Fiber: grelha, componentes, fios
│  ├─ ui/              # React: barra, menus, painéis
│  └─ persistence/     # JSON + Zod
├─ public/models/      # modelos 3D .glb
├─ assets-src/         # ficheiros .blend (Git LFS)
├─ tests/e2e/          # testes Playwright
├─ docs/               # dossier, decisões, manual
└─ CLAUDE.md           # regras do projeto para o Claude Code
```

Correspondência com os módulos do dossier (secção 9):

Módulo do dossier	Pasta
Gestão da UI, Painel de propriedades	src/ui
Biblioteca de componentes	src/core/library + src/ui
Drag & Drop, Gestor da grelha, Seleção, Sistema de terminais, Gestor de fios, Feedback visual	src/scene + src/store
Grafo do circuito	src/core/graph
Gerador de netlist	src/core/netlist
Validação	src/core/validation
Motor de cálculo	src/core/solver
Guardar/Carregar	src/persistence
Testes	ficheiros *.test.ts + tests/e2e

Motor de cálculo

Método recomendado: **análise nodal modificada (MNA)**, a mesma base do SPICE e do LTspice. Ela trata de uma só vez série, paralelo, circuitos mistos e várias fontes. Não é preciso escrever casos especiais para "série" ou "paralelo".

O princípio: cada nó (exceto o GND) tem uma tensão desconhecida, e cada fonte de tensão tem uma corrente desconhecida. Cada componente acrescenta valores numa matriz (o "stamp"). Depois resolve-se um sistema linear:

```
\begin{bmatrix} G & B \\ C & D \end{bmatrix} \begin{bmatrix} v \\ i \end{bmatrix} = \begin{bmatrix} i_s \\ e \end{bmatrix}
```

G contém as condutâncias (1/R), B e C ligam as fontes de tensão aos nós, v são as tensões dos nós e i as correntes das fontes. A resolução usa eliminação de Gauss com pivotagem parcial.

Fases do motor, pela ordem do dossier:

1. **DC linear**: resistência, fonte DC, GND, interruptor (resistência muito baixa ou circuito aberto), lâmpada como resistência. Resultados U, I, P por componente.
2. **Validação antes do cálculo**: sem fonte, sem GND, terminais soltos, curto direto na fonte, valores inválidos. Matriz singular = mensagem clara ao utilizador.
3. **Díodo e LED**: modelo não linear, resolvido por Newton-Raphson (várias iterações da MNA). Começar por um modelo simples (tensão de limiar + resistência).
4. **Condensador e bobina em transitório**: método de Euler implícito, passo de tempo fixo, gráfico do resultado.
5. **AC**: números complexos e impedâncias, depois osciloscópio virtual.

Cada fase só começa quando a anterior passa nos testes. Os testes de referência são circuitos calculados à mão: divisor de tensão, resistências em paralelo, circuito de demonstração 12 V + resistência + LED. Tolerância numérica típica: 1e-9.

GitHub

Sim, o GitHub é necessário. É um projeto de grupo e com IA a escrever código: sem histórico de versões, um erro pode apagar dias de trabalho. O GitHub também serve para dividir tarefas e publicar a app online de graça.

O que montar:

- **Um repositório** (ex.: simulador-geii), com todos os membros do grupo como colaboradores.

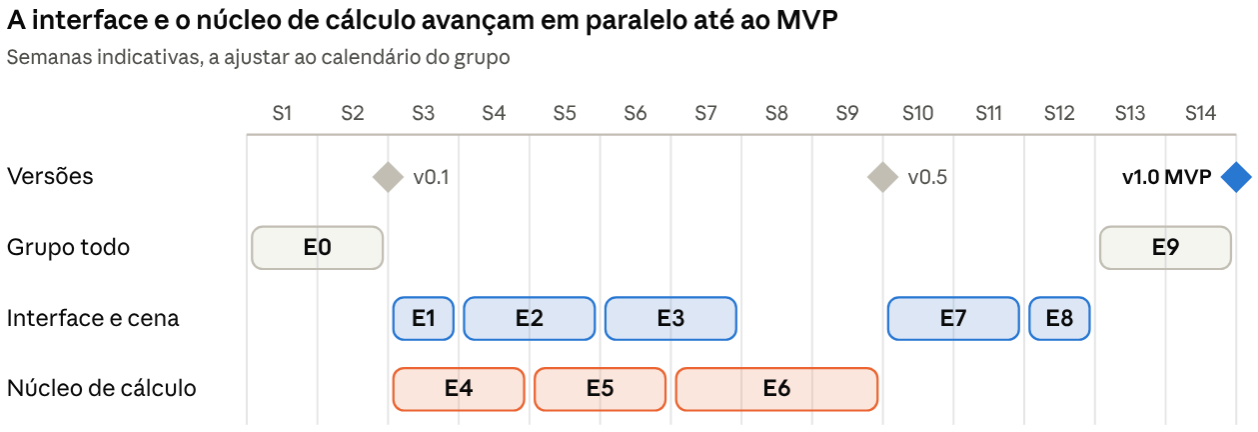
- **Ramo main protegido**: ninguém escreve diretamente nele. Cada tarefa tem o seu ramo (feature/drag-drop, fix/fios-soltos…).
- **Issues**: uma issue por tarefa, com critério de "feito" claro.
- **GitHub Projects**: quadro com colunas A fazer / Em curso / Em revisão / Feito.
- **Pull requests**: cada ramo volta ao main por PR, revisto por outro membro.
- **GitHub Actions**: a cada PR, executa lint + testes + build. Se o solver falhar um teste, o PR fica vermelho.
- **GitHub Pages**: publicação automática da versão main, com um link para mostrar aos professores.
- **Git LFS** para os ficheiros .blend e .glb grandes.
- **Releases com tags** (v0.1, v0.2…) no fim de cada etapa.

Mensagens de commit curtas e no mesmo formato, por exemplo: feat(solver): adiciona fontes de tensão; fix(fios): fios seguem o componente ao mover.

Uma dica: pede o **GitHub Student Developer Pack** com o teu email da universidade. Dá acesso gratuito ao Copilot e a outras ferramentas.

Plano etapa por etapa

O MVP cabe em cerca de 14 semanas se a interface e o núcleo de cálculo forem feitos em paralelo. Isto é possível porque o núcleo não depende da interface: é testado sozinho com Vitest. As etapas seguem a numeração da secção 16 do dossier.



plano do MVP · 14 semanas indicativas

Três versões marcam o caminho: v0.1 = ambiente pronto e app vazia online; v0.5 = o solver resolve os circuitos de referência nos testes; v1.0 = demonstração completa do dossier (secção 20).

1. **E0 – Definição e instalação** (S1–S2)

- Fechar as decisões da secção 22 do dossier: nome, lista final do MVP, fios ortogonais, câmara.
- Instalar o ambiente, criar o repositório, o CLAUDE.md e a CI.
- Feito quando: todos fazem clone, pnpm dev abre a app vazia, e o link GitHub Pages funciona.

2. **E1 – Layout** (S3)
   - Barra superior, menu recolhível, zona livre com câmara ortográfica e grelha, painéis vazios.
   - Feito quando: a zona livre ocupa a maior parte do ecrã e os painéis abrem e fecham.
3. **E2 – Interação** (S4–S5)
   - Arrastar do menu para a zona livre, snap, seleção, rodar, duplicar, apagar, zoom, pan, desfazer/refazer.
   - Para começar, componentes como caixas simples; os modelos 3D chegam depois.
4. **E3 – Ligações** (S6–S7)
   - Terminais clicáveis, criação de fios ortogonais, fios que seguem o componente, apagar fios.
   - Opção de debug: mostrar o número do nó em cada terminal.
5. **E4 – Dados** (S3–S4, em paralelo)
   - Tipos TypeScript: Component, Terminal, Wire, Project. Biblioteca: fonte DC, GND, resistência, interruptor, LED, lâmpada.
   - Painel de propriedades ligado ao modelo (valor, unidade, rotação).
6. **E5 – Circuito** (S5–S6)
   - Grafo (terminais → nós), netlist, validações da secção 13 do dossier.
   - Feito quando: os testes detetam circuito aberto, curto, falta de GND e falta de fonte.
7. **E6 – Cálculo DC** (S7–S9)
   - Solver MNA com Gauss, resultados U, I, P por componente.
   - Feito quando: divisor de tensão, paralelo e circuito misto dão o mesmo resultado que o cálculo à mão. Versão v0.5.
8. **E7 – Feedback e integração** (S10–S11)
   - Botão Simular ligado ao solver, painel de resultados, lâmpada com brilho proporcional à potência, LED aceso, mensagens de erro claras.
   - Modelos 3D do Blender substituem as caixas simples.
9. **E8 – Persistência** (S12)

    - Novo, guardar e abrir projeto em JSON, com número de versão do formato e verificação Zod.
10. **E9 – Testes e demonstração** (S13–S14)
    - Testes Playwright do percurso completo, correção de bugs, teste com colegas que não conhecem a app.
    - Verificar os critérios de aceitação da secção 19. Versão v1.0.
11. **E10 – Evolução** (depois do MVP)
    - Por esta ordem: instrumentos (voltímetro, amperímetro), díodo real, condensador e bobina, AC e osciloscópio, 3D completo, VR com WebXR.

Trabalhar com Claude, Claude Code e outras IAs

Cada IA tem um papel. Tu continuas a ser o responsável: tens de perceber e saber explicar cada parte, porque vais apresentar o projeto.

Ferramenta	Papel no projeto	Exemplo de pedido
Claude (esta conversa)	Arquitetura, explicar conceitos, preparar tarefas, rever código, documentação e apresentação	"Explica-me a MNA com um divisor de tensão"
Claude Code	Escrever o código no repositório, executar testes, corrigir erros, fazer commits	"Implementa a issue #12 e escreve os testes"
GitHub Copilot	Autocompletar enquanto escreves no VS Code	Completar uma função pequena
ChatGPT ou Gemini	Segunda opinião, verificar um cálculo, outra explicação	"Este resultado de 9,6 mA está certo?"
IA de imagem (opcional)	Ícones e imagens para o menu e a apresentação	Ícone de resistência em estilo plano

Ciclo de trabalho para cada tarefa:

1. Criar a issue no GitHub com o objetivo e o critério de "feito" (o Claude ajuda a escrevê-la).
2. Criar o ramo: git switch -c feature/nome-da-tarefa.
3. No Claude Code, pedir primeiro um **plano** (modo plan), ler e corrigir o plano.
4. Deixar o Claude Code implementar **com testes**, em passos pequenos.

5. Testar tu mesmo na app (pnpm dev) e ler o código. Se não perceberes uma parte, pede uma explicação.
6. Abrir o pull request; outro membro do grupo revê; a CI tem de ficar verde.
7. Fazer merge no main e mover o cartão no quadro do GitHub Projects.

Regras para a IA não sair do controlo:

- Uma tarefa pequena de cada vez (1 a 3 horas de trabalho), nunca "faz a app toda".
- O ficheiro **CLAUDE.md** na raiz descreve o projeto, a stack, as pastas e as regras. O Claude Code lê-o em cada sessão.
- Nenhuma alteração ao solver sem teste com um circuito calculado à mão.
- Commit frequente: se a IA estragar algo, volta-se atrás com o Git.
- Guardar as decisões importantes em docs/decisoes.md, para o grupo e para as próximas sessões de IA.

Organização do grupo e acompanhamento

O acompanhamento faz-se em dois sítios: o **quadro GitHub Projects** para as tarefas do dia a dia, e a **planilha Excel** (enviada com este documento) para a visão geral por etapa.

A planilha tem três folhas: Etapas (cada tarefa com responsável, semana, estado e critério de "feito"), Instalação (lista do ambiente a verificar) e Testes de referência (circuitos com o resultado teórico a comparar com o simulador).

Distribuição sugerida, a partir da secção 17 do dossier:

Função	Pastas principais	Etapas
Coordenação e integração	CLAUDE.md, docs, CI	E0, E7, E9
UI/UX	src/ui	E1, E4 (painel)
Interação e cena	src/scene, src/store	E2, E3
Conectividade e motor de cálculo	src/core	E4, E5, E6
Assets 3D	assets-src, public/models	E7
Testes e documentação	tests, docs	E5, E6, E9

Ritmo: uma reunião curta por semana (15–30 min). Cada um diz o que fez, o que vai fazer e o que o bloqueia. No fim de cada etapa: tag de versão no GitHub e atualização da planilha.

Riscos e regras de ouro

O maior risco é o escopo: terminar o MVP DC antes de tocar em AC, 3D completo ou VR.

Risco	Medida com esta stack
A IA escreve código que ninguém percebe	Tarefas pequenas, revisão em PR, pedir explicações ao Claude
Resultados elétricos errados	Testes Vitest com circuitos calculados à mão, CI obrigatória
Fios não seguem os componentes	Fios guardam IDs de terminais, não posições; a posição é calculada
Matriz singular (circuito sem solução)	Validação antes do solver e mensagem clara ao utilizador
Projetos guardados deixam de abrir	Campo version no JSON e verificação Zod
Modelos 3D pesados	Low-poly, formato .glb comprimido
Conflitos entre membros no Git	Um ramo por tarefa, merges pequenos e frequentes

Regras de ouro:

- O núcleo (src/core) nunca importa React nem Three.js.
- Nada entra no main sem testes a passar.
- Primeiro funcional com caixas simples, depois bonito com modelos 3D.

Prompt para o Claude Code

Abre um terminal na pasta onde queres o projeto, inicia o Claude Code e cola o texto abaixo. Ele verifica o teu computador, diz o que instalar e monta a base do projeto, parando no fim de cada passo para tu confirmares.

```
Contexto
Sou estudante de BUT GEII (3.º ano). Vou desenvolver em grupo um "Simulador
2.5D de Circuitos GEII": um simulador pedagógico de circuitos elétricos com
uma grande zona livre tipo LTspice, componentes com aspeto 3D (câmara
ortográfica) e um motor de cálculo elétrico. O MVP faz apenas DC: fonte DC,
GND, resistência, interruptor, LED e lâmpada; resultados U, I, R e P; deteção
de circuito aberto, curto-circuito e falta de GND; guardar e abrir projetos
em JSON. Mais tarde: díodo real, condensador, bobina, AC, 3D completo e VR
(WebXR).
```

```
Stack decidida (não mudes sem me perguntar)
- TypeScript em modo strict, React, Vite, pnpm
- Tailwind CSS para a interface
- Three.js com React Three Fiber e @react-three/drei para a cena 2.5D
- Zustand (+ zundo) para o estado e o desfazer/refazer
- Zod para validar o JSON dos projetos
- Motor de cálculo próprio em TypeScript puro: análise nodal modificada (MNA)
com eliminação de Gauss e pivotagem parcial
- Vitest (testes unitários), Playwright (testes E2E), ESLint + Prettier
- GitHub: repositório, GitHub Actions (lint + testes + build), deploy em
GitHub Pages

Faz o trabalho passo a passo e espera pela minha confirmação no fim de cada
passo.

Passo 1 – Verificar o meu computador
Deteta o sistema operativo e verifica se tenho: Git, Node.js LTS, pnpm, VS
Code, GitHub CLI (gh) e Git LFS. Mostra uma tabela: ferramenta, versão
encontrada ou "falta", e o comando exato para instalar o que falta no meu
sistema. Diz também que extensões do VS Code devo instalar. Não instales nada
sem me perguntar.

Passo 2 – Criar o projeto
Cria o projeto Vite + React + TypeScript chamado simulador-geii, instala as
dependências da stack e configura Tailwind, ESLint, Prettier, Vitest e
Playwright. Cria esta estrutura de pastas:
src/core/{model,library,graph,netlist,validation,solver}, src/store,
src/scene, src/ui, src/persistence, public/models, assets-src, tests/e2e,
docs.
Regra: src/core nunca importa React nem Three.js. Configura uma regra ESLint
que o impeça.

Passo 3 – CLAUDE.md e documentação
Cria um CLAUDE.md com: descrição do projeto, stack, estrutura das pastas,
comandos (dev, test, lint, build) e regras (tarefas pequenas, testes
obrigatórios para qualquer alteração do solver, TypeScript strict, mensagens
de commit no formato feat|fix|docs(escopo): descrição). Cria também um
README.md em português e um ficheiro docs/decisoes.md.

Passo 4 – Git e GitHub
Inicializa o Git com um .gitignore adequado e Git LFS para *.blend e *.glb.
Explica-me como criar o repositório no GitHub (com gh, se estiver instalado),
fazer o primeiro push, proteger o ramo main e adicionar os meus colegas. Cria
o workflow GitHub Actions (lint, testes, build) e o workflow de deploy para
GitHub Pages, com o base path correto no Vite.
```

Passo 5 – Primeira prova de que tudo funciona
- Layout vazio: barra superior, menu de componentes recolhível à esquerda, zona livre ao centro com câmara ortográfica e grelha (React Three Fiber), painel de propriedades à direita, painel de resultados em baixo.
- Tipos base em src/core/model (Component, Terminal, Wire, Project).
- Primeira versão do solver MNA DC, só com resistências e fontes de tensão, com testes Vitest:
  a) divisor de tensão: 12 V, R1 = 1 kΩ em cima, R2 = 2 kΩ em baixo → nó do meio = 8 V, I = 4 mA;
  b) duas resistências de 1 kΩ em paralelo sob 10 V → corrente total = 20 mA.
- Executa pnpm test e pnpm build e mostra-me o resultado.

No fim, faz um resumo: o que foi feito, como arranco a app (pnpm dev) e as 5 próximas tarefas sugeridas para as etapas E1 e E2, cada uma escrita como uma issue do GitHub (título, descrição, critério de "feito").

Explica tudo em português simples: estou a aprender e preciso de perceber o que fazes para apresentar o projeto.